

# 10강. 쇼핑몰 프로젝트(상품)



# 쇼핑몰 프로젝트

## ■ 프로젝트 개요

프로젝트 이름 : shopping

개발 환경: React + TypeScript + Vite

제품요구사항:

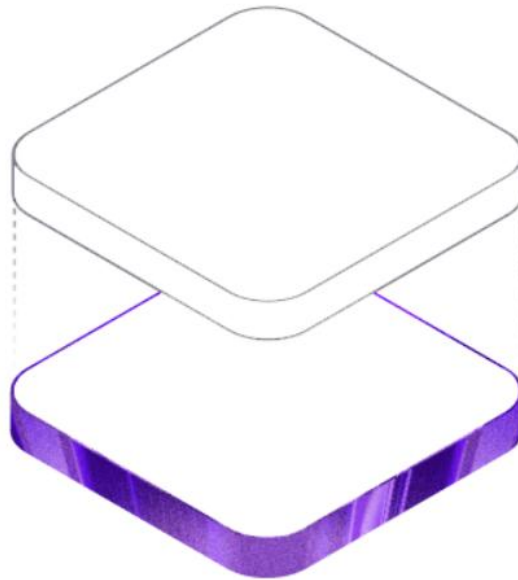
- 상품: 상품 목록, 상품 정보, 상품 등록
- 회원: 회원 가입, 로그인, 관리자 대시보드

# 쇼핑몰 프로젝트

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

## 컴퓨터 기기 쇼핑몰

최신 컴퓨터 기기와 액세서리를 만나보세요.



# 쇼핑몰 프로젝트

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

## 상품 목록



무선 마우스

가격: 32000원



무선 키보드

가격: 45000원



USB 메모리

가격: 15000원



모니터

가격: 200000원

# 쇼핑몰 프로젝트

## 무선 마우스



무선 마우스에 대한 설명입니다.

**가격: 32000원**

목록으로

장바구니에 추가

# 쇼핑몰 프로젝트

Home 상품 목록 상품 등록 로그인

상품 등록

상품명

상품명을 입력하세요.

가격

0

설명

설명을 입력하세요.

이미지

파일 선택

선택된 파일 없음

등록

# 쇼핑몰 프로젝트

Home 상품 목록 상품 등록 로그인

로그인

아이디 아이디를 입력하세요.

비밀번호 비밀번호를 입력하세요.

로그인

# 쇼핑몰 프로젝트

## ▪ App.tsx

```
function App() {  
  return (  
    <>  
      <section className="app">  
        <BrowserRouter>  
          <Header />  
          <Routes>  
            <Route path="/" element={<Home />} />  
            <Route path="/products" element={<ProductList />} />  
            <Route path="/products/:id" element={<ProductInfo />} />  
            <Route path="/products/add" element={<AddProduct />} />  
            <Route path="/signin" element={<SignIn />} />  
          </Routes>  
        </BrowserRouter>  
      </section>  
    </>  
  )  
}
```



# 쇼핑몰 프로젝트

## ▪ App.css

```
/* Header 스타일 */  
.header {  
  height: 50px;  
  margin-bottom: 20px;  
  background-color: #333;  
  padding: 10px 0;  
  text-align: center;  
}  
.header a {  
  margin: 0 16px;  
  text-decoration: none;  
  color: orange;  
  line-height: 50px;  
}  
.header a:hover {color: white;}
```

# 쇼핑몰 프로젝트

## ▪ App.css

```
/* 상품 목록 스타일 */
.product-list {width: 80%; margin: 20px auto;}
.product-list h2 {margin-bottom: 20px;}
.card-container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
.card-container a {text-decoration: none;}
.card {
  width: 250px;
  border: 1px solid #ccc;
  border-radius: 5px;
  margin: 10px;
}
.product-image {width: 100%; height: 200px;}
.card-title {
  font-size: 18px;
  margin: 10px 0;
  color: #333;
}
.card-text {font-size: 16px; color: #555;}
```

# 쇼핑몰 프로젝트

## ■ App.css

```
/* 상품 상세 페이지 스타일 */
.product-info {
  width: 80%;
  max-width: 900px;
  margin: 40px auto;
  padding: 30px;
  border-radius: 12px;
}
.product-info h2 {
  margin-bottom: 24px;
  font-size: 28px;
  color: #222;
  border-bottom: 2px solid #e91e63;
  padding-bottom: 12px;
}
.product-details {
  display: flex;
  align-items: flex-start;
  justify-content: center;
  gap: 40px;
}
```

```
.product-details img {
  width: 320px;
  height: 320px;
  border-radius: 10px;
  padding: 16px;
  border: 1px solid #eee;
}
.product-content {
  max-width: 420px;
  display: flex;
  flex-direction: column;
  gap: 16px;
}
.product-content .price {
  font-size: 24px;
  color: #e91e63;
  font-weight: bold;
  margin-top: 8px;
  background-color: #fce4ec;
  padding: 10px 16px;
  border-radius: 8px;
  display: inline-block;
}
```

# 쇼핑몰 프로젝트

## ■ App.css

```
.product-buttons {  
  display: flex;  
  gap: 12px;  
  margin-top: 8px;  
}  
  
.product-buttons .btn-list {  
  padding: 10px 24px;  
  border: 2px solid #555;  
  border-radius: 8px;  
  background-color: #fff;  
  color: #555;  
  font-size: 15px;  
  font-weight: 600;  
  cursor: pointer;  
  transition: all 0.2s;  
}  
  
.product-buttons .btn-list:hover {  
  background-color: #555;  
  color: #fff;  
}
```

```
.product-buttons .btn-cart {  
  padding: 10px 24px;  
  border: none;  
  border-radius: 8px;  
  background-color: #e91e63;  
  color: #fff;  
  font-size: 15px;  
  font-weight: 600;  
  cursor: pointer;  
  transition: all 0.2s;  
}  
  
.product-buttons .btn-cart:hover {  
  background-color: #c2185b;  
}
```

# 쇼핑몰 프로젝트

## ▪ App.css

```
/* 상품 등록 스타일 */  
.add-product {  
  width: 400px;  
  margin: 20px auto;  
  padding: 20px;  
  border: 1px solid #ccc;  
  border-radius: 5px;  
}  
.add-product h2 {  
  text-align: center;  
  margin-bottom: 20px;  
  color: #333;  
}  
.add-product form {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
}
```

# 쇼핑몰 프로젝트

## ▪ App.css

```
.add-product label {font-size: 16px; color: #555;}
.add-product input {
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.add-product button {
  padding: 10px;
  background-color: #e91e63;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
.add-product button:hover {background-color: #d81b60;}
```

# 쇼핑몰 프로젝트

## ▪ App.css

```
/* 로그인 스타일 */  
.signin {  
  width: 400px;  
  margin: 20px auto;  
  padding: 20px;  
  border: 1px solid #ccc;  
  border-radius: 5px;  
}  
.signin h2 {  
  text-align: center;  
  margin-bottom: 20px;  
  color: #333;  
}  
.signin form {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
}
```

# 쇼핑몰 프로젝트

## ▪ App.css

```
.signin label {font-size: 16px; color: #555;}
.signin input {
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.signin button {
  padding: 10px;
  background-color: #0070c0;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
.signin button:hover {background-color: #005596;}
```



# 쇼핑몰 프로젝트

## ▪ layouts/Header.tsx

```
const Header = () => {  
  return (  
    <header className="header">  
      <nav>  
        <NavLink to="/">Home</NavLink>  
        <NavLink to="/products">상품 목록</NavLink>  
        <NavLink to="/products/add">상품 등록</NavLink>  
        <NavLink to="/signin">로그인</NavLink>  
      </nav>  
    </header>  
  )  
}  
  
export default Header
```

# 쇼핑몰 프로젝트

## ▪ layouts/Home.tsx

```
import mainImage from '../assets/hero.png'

const Home = () => {
  return (
    <div className="home">
      <h2>컴퓨터 기기 쇼핑몰</h2>
      <section>
        <h3>최신 컴퓨터 기기와 액세서리를 만나보세요.</h3>
        <img src={mainImage} alt="메인 이미지" />
      </section>
    </div>
  )
}
```

# 쇼핑몰 프로젝트

## ▪ data/products.json

```
[  
  {  
    "id": 1,  
    "name": "무선 마우스",  
    "price": 32000,  
    "description": "무선 마우스에 대한 설명입니다.",  
    "image": "mouse.png"  
  },  
  {  
    "id": 2,  
    "name": "무선 키보드",  
    "price": 45000,  
    "description": "무선 키보드에 대한 설명입니다.",  
    "image": "keyboard.png"  
  },  
]
```

# 쇼핑몰 프로젝트

## ▪ data/products.json

```
{  
  "id": 3,  
  "name": "USB 메모리",  
  "price": 15000,  
  "description": "USB 메모리에 대한 설명입니다.",  
  "image": "usb.png"  
},  
{  
  "id": 4,  
  "name": "모니터",  
  "price": 200000,  
  "description": "모니터에 대한 설명입니다.",  
  "image": "monitor.png"  
}  
]
```

# 쇼핑몰 프로젝트

## ▪ data/users.json

```
import { Link } from 'react-router-dom'
import products from '../data/products.json'
import mouse from '../assets/mouse.png'
import keyboard from '../assets/keyboard.png'
import monitor from '../assets/monitor.png'
import usb from '../assets/usb.png'

// 이미지 파일과 상품 데이터의 매핑을 위한 객체입니다.
const imageMap: Record<string, string> = {
  'mouse.png': mouse,
  'keyboard.png': keyboard,
  'monitor.png': monitor,
  'usb.png': usb,
}
```

# 쇼핑몰 프로젝트

## ■ products/ProductList.tsx

```
const ProductList = () => {  
  return (  
    <div className="product-list">  
      <h2>상품 목록</h2>  
      <div className='card-container'>  
        {products.map((product: any) => (  
          <Link to={` /products/${product.id}`}>  
            <div key={product.id} className="card">  
              <img  
                src={imageMap[product.image]}  
                alt={product.name}  
                className="product-image"  
              />  
              <div className='card-body'>  
                <h3 className='card-title'>{product.name}</h3>  
                <p className='card-text'>가격: {product.price}원</p>  
              </div>  
            </div>  
          </Link>  
        ))}  
      </div>  
    </div>  
  )  
}
```

# 쇼핑몰 프로젝트

## ▪ products/ProductInfo.tsx

```
import { useParams } from 'react-router-dom'
import products from '../data/products.json'
import { imageMap } from './ProductList'

const ProductInfo = () => {
  const { id } = useParams() // URL에서 상품 ID를 가져옵니다.

  // products 배열에서 해당 ID를 가진 상품을 찾습니다.
  // id는 문자열이므로 Number로 변환하여 비교합니다.
  const product = products.find((p: any) => p.id === Number(id))

  if (!product) {
    return <p>상품을 찾을 수 없습니다.</p>
  }
}
```

# 쇼핑몰 프로젝트

## ▪ products/ProductInfo.tsx

```
return (  
  <div className="product-info">  
    <h2>{product.name}</h2>  
    <div className='product-details'>  
      <img  
        src={imageMap[product.image]}  
        alt={product.name}  
        className="product-image"  
      />  
      <div className="product-content">  
        <p>{product.description}</p>  
        <p className='price'>가격: {product.price}원</p>  
        <div className="product-buttons">  
          <button  
            className="btn-list"  
            onClick={() => navigate('/products')}  
          >  
            목록으로  
          </button>  
          <button className="btn-cart">장바구니에 추가</button>  
        </div>  
      </div>  
    </div>  
  </div>  
)
```



# 쇼핑몰 프로젝트

## products/AddProduct.tsx

```
/* 상품 추가를 위한 폼 데이터 인터페이스입니다.  
interface File은 브라우저에서 제공하는 타입으로,  
파일 입력에서 사용됩니다.*/
```

```
interface FormData {  
  name: string  
  price: number  
  description: string  
  image: File | null  
}
```

파일을 선택하지 않으면 null이고,  
선택하면 File이 입력됨

```
const AddProduct = () => {  
  // 상품 추가를 위한 폼 데이터를 관리 상태.  
  const [formData, setFormData] = useState<FormData>({  
    name: '',  
    price: 0,  
    description: '',  
    image: null  
  })
```

```
  const navigate = useNavigate();
```

# 쇼핑몰 프로젝트

## products/AddProduct.tsx

```
// 입력값 변경
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const { name, value, files } = e.target
  // 파일 입력인 경우, 파일 객체를 상태에 저장합니다.
  // 일반 텍스트 입력인 경우, 해당 값을 상태에 저장합니다.
  if (name === 'image' && files) {
    setFormData({ ...formData, image: files[0] })
  } else {
    setFormData({ ...formData, [name]: value })
  }
}

// 폼 제출
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();

  // 입력값 검증
  if (!formData.name || !formData.price || !formData.description) {
    alert('모든 필드를 입력해주세요.');
```

# 쇼핑몰 프로젝트

## products/AddProduct.tsx

```
if (formData.price <= 0) { //가격 검증
  alert('가격은 0보다 커야 합니다.');
```

```
  return;
}
```

```
navigate('/products'); // 상품 추가 후 목록 페이지로 이동
}
```

```
return (
  <div className="add-product">
    <h2>상품 등록</h2>
    <form onSubmit={handleSubmit}>
      <div>
        <label htmlFor="name">상품명</label>
        <input
          type="text"
          id="name"
          name="name"
          placeholder="상품명을 입력하세요."
          value={formData.name}
          onChange={handleChange}
        />
      </div>
    </form>
  </div>
);
```

# 쇼핑몰 프로젝트

## ▪ products/AddProduct.tsx

```
<div>
  <label htmlFor="price">가격</label>
  <input
    type="number"
    id="price"
    name="price"
    placeholder="가격을 입력하세요."
    value={formData.price}
    onChange={handleChange}
  />
</div>
<div>
  <label htmlFor="description">설명</label>
  <input
    type="text"
    id="description"
    name="description"
    placeholder="설명을 입력하세요."
    value={formData.description}
    onChange={handleChange}
  />
</div>
```

# 쇼핑몰 프로젝트

- products/AddProduct.tsx

```
<div>
  <label htmlFor="image">이미지</label>
  <input
    type="file"
    id="image"
    name="image"
    onChange={handleChange}
  />
</div>
<button type='submit'>등록</button>
</form>
</div>
)
```

# 쇼핑몰 프로젝트

## ■ 프로젝트 개요

프로젝트 이름 : shopping

개발 환경: React + TypeScript + Vite

제품요구사항:

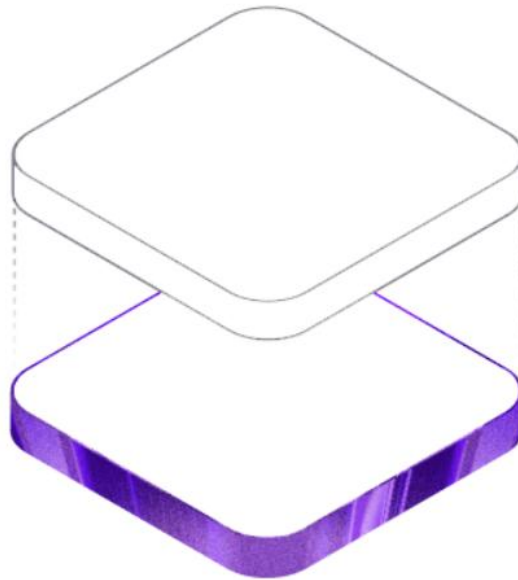
- 상품: 상품 목록, 상품 정보, 상품 등록
- 회원: 회원 가입, 로그인, 관리자 대시보드

# 쇼핑몰 프로젝트

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

## 컴퓨터 기기 쇼핑몰

최신 컴퓨터 기기와 액세서리를 만나보세요.



# 쇼핑몰 프로젝트

## ▪ data/users.json

```
[  
  {  
    "id": 1,  
    "username": "admin",  
    "password": "admin123",  
    "role": "admin"  
  },  
  {  
    "id": 2,  
    "username": "user1",  
    "password": "user123",  
    "role": "user"  
  },  
  {  
    "id": 3,  
    "username": "user2",  
    "password": "user123",  
    "role": "user"  
  }  
]
```



# 쇼핑몰 프로젝트

## ▪ users/SignIn.tsx

```
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import users from '../data/users.json'

// 로그인 폼 데이터 타입 정의
interface SignInForm {
  username: string
  password: string
}

const SignIn = () => {
  const [formData, setFormData] = useState<SignInForm>({
    username: '',
    password: '',
  })
  // 로그인 결과 상태 관리 (성공/실패 메시지 등)
  const [loginResult, setLoginResult] = useState<string>('')

  const navigate = useNavigate();
```

# 쇼핑몰 프로젝트

## ▪ users/SignIn.tsx

```
// 입력 필드 변경 핸들러
const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const { name, value } = e.target
  setFormData({ ...formData, [name]: value })
}

// 폼 제출 핸들러
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault()
  // 입력값 추출
  const { username, password } = formData

  // 간단한 유효성 검사
  if (!username || !password) {
    alert('아이디와 비밀번호를 모두 입력해주세요.')
    return
  }
}
```

# 쇼핑몰 프로젝트

## ▪ users/SignIn.tsx

```
// 로그인 일치 여부
const user = users.find(user =>
  user.username === username && user.password === password)
if (!user) {
  alert('아이디 또는 비밀번호가 올바르지 않습니다.')
  setLoginResult('fail') // 로그인 실패 상태 업데이트
  return
}
console.log('로그인 시도:', formData)

if (user.role === 'admin') {
  setLoginResult('success') // 로그인 성공 상태 업데이트
  // 관리자 로그인 시 대시보드로 이동하면서 사용자 정보 전달
  navigate('/dashboard', { state: { username: user.username, role: user.role } })
} else {
  setLoginResult('success')
  navigate('/products')
}
}
```

# 쇼핑몰 프로젝트

## ▪ users/SignIn.tsx

```
return (  
  <div className='signin'>  
    <h2>로그인</h2>  
    <form onSubmit={handleSubmit}>  
      <div>  
        <label htmlFor="username">아이디</label>  
        <input  
          type="text"  
          id="username"  
          name="username"  
          placeholder="아이디를 입력하세요."  
          value={formData.username}  
          onChange={handleChange} />  
      </div>  
      <div>  
        <label htmlFor="password">비밀번호</label>  
        <input  
          type="password"  
          id="password"
```

# 쇼핑몰 프로젝트

## ▪ users/SignIn.tsx

```
        name="password"
        placeholder="비밀번호를 입력하세요."
        value={formData.password}
        onChange={handleChange} />
      </div>
      <button type="submit">로그인</button>
    </form>

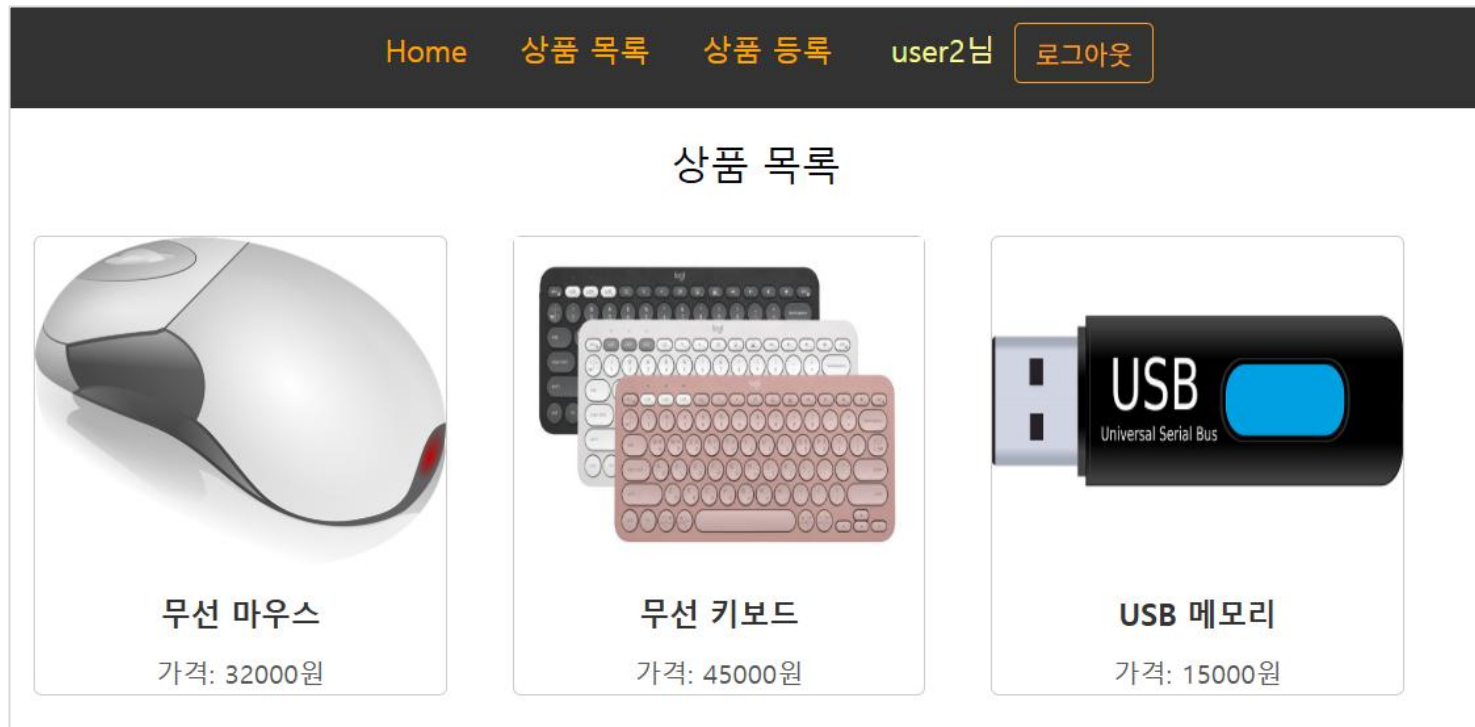
    {loginResult === 'fail' && <p className="error">로그인 실패. 다시 시도해주세요.</p>}
  </div>
)
}

export default SignIn
```

```
.error {
  color: ■rgb(197, 13, 13);
  text-align: center;
  margin-top: 10px;
}
```

# 쇼핑몰 프로젝트

## 로그인 / 로그아웃 구현



# 쇼핑몰 프로젝트

## 로그인 / 로그아웃 구현 - App.tsx

```
function App() {  
  // 로그인 상태 관리  
  const [isLoggedIn, setIsLoggedIn] = useState(false);  
  
  // 로그인한 사용자 ID 관리 (예: username)  
  const [userId, setUserId] = useState<string | null>(null);  
  
  // 로그인 상태 업데이트 함수  
  const handleLogin = (username: string) => {  
    setIsLoggedIn(true);  
    setUserId(username);  
  }  
  
  // 로그아웃 함수  
  const handleLogout = () => {  
    setIsLoggedIn(false);  
    setUserId(null);  
  }  
}
```

# 쇼핑몰 프로젝트

## 로그인 / 로그아웃 구현 - App.tsx

```
return (  
  <>  
    <section className="app">  
      <BrowserRouter>  
        <Header isLoggedIn={isLoggedIn} userId={userId} onLogout={handleLogout} />  
        <Routes>  
          <Route path="/" element={<Home />} />  
          <Route path="/products" element={<ProductList />} />  
          <Route path="/products/:id" element={<ProductInfo />} />  
          <Route path="/products/add" element={<AddProduct />} />  
          <Route path="/signin" element={<SignIn onLogin={handleLogin} />} />  
        </Routes>  
      </BrowserRouter>  
    </section>  
  </>  
)  
}
```



# 쇼핑몰 프로젝트

## 로그인 / 로그아웃 구현 - Header.tsx

// Header 컴포넌트 Props 타입 정의

```
interface HeaderProps {  
  isLoggedIn: boolean;  
  userId?: string | null;  
  onLogout: () => void;  
}
```

로그인 전에는 null, 로그인 후에는  
사용자 이름 문자열이 들어온다.  
?는 props 자체가 전달되지 않을 때  
도 있음

```
const Header = ({ isLoggedIn, userId, onLogout }: HeaderProps) => {  
  const navigate = useNavigate();
```

```
  return (  
    <header className="header">  
      <nav>  
        <NavLink to="/">Home</NavLink>  
        <NavLink to="/products">상품 목록</NavLink>  
        <NavLink to="/products/add">상품 등록</NavLink>
```

# 쇼핑몰 프로젝트

## 로그인 / 로그아웃 구현 - Header.tsx

```
{isLoggedIn ? (  
  <div className="header-user-info">  
    <span>{userId}님</span>  
    <button  
      onClick={() => {  
        onLogout();  
        navigate('/');  
      }}  
      className="logout-btn"  
    >  
      로그아웃  
    </button>  
  </div>  
) : (  
  <NavLink to="/signin">로그인</NavLink>  
)  
</nav>  
</header>  
)  
}
```

# 쇼핑몰 프로젝트

## 로그인 / 로그아웃 구현 – App.css

```
.header .header-user-info {  
  display: inline-flex;  
  align-items: center;  
  gap: 12px;  
  color: #c9c98e;  
  margin-left: 20px;  
}  
  
.header .logout-btn {  
  margin-top: 4px;  
  padding: 6px 12px;  
  border: 1px solid #ff9f1c;  
  border-radius: 4px;  
  background-color: transparent;  
  color: #ff9f1c;  
  cursor: pointer;  
}  
  
.header .logout-btn:hover {background-color: #ff9f1c; color: #333;}
```

# 쇼핑몰 프로젝트

## 로그인 / 로그아웃 구현 – SignIn.tsx

```
// 로그인 컴포넌트 Props 타입 정의
interface SignInProps {
  | onLogin: (username: string) => void;
}

const SignIn = ({ onLogin }: SignInProps) => {
  const [formData, setFormData] = useState<SignInForm>({
    username: '',
    password: '',
  })

  // 로그인 일치 여부
  const user = users.find(user =>
    user.username === username && user.password === password)
  if (!user) {
    alert('아이디 또는 비밀번호가 올바르지 않습니다.')
    setLoginResult('fail') // 로그인 실패 상태 업데이트
    return
  }

  console.log('로그인 시도:', formData)
  onLogin(user.username); // 로그인 성공 시 부모 컴포넌트에 알림
}
```

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

### 로그인

아이디

아이디를 입력하세요.

비밀번호

비밀번호를 입력하세요.

로그인

[아직 계정이 없으신가요? 회원가입](#)

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

### 회원가입

아이디:

이메일:

비밀번호:

비밀번호 확인:

[회원가입](#)

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현 – users/SignIn.tsx

```
    <button type="submit">로그인</button>
  </form>
  {/* 아직 계정이 없으신가요? 회원가입 링크 추가 가능 */}
  <p className='signup-link'>
    아직 계정이 없으신가요? <Link to="/signup">회원가입</Link>
  </p>

  {/* 로그인 실패 메시지 */}
  {loginResult === 'fail' && <p className="error">로그인 실패. 다시 시도해주세요.</p>}}
</div>
)
```

```
.signup-link {
  text-align: center;
  margin-top: 15px;
}

.signup-link a {
  color: #640aec;
  text-decoration: none;
}

.signup-link a:hover {text-decoration: underline;}
```

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현 – users/SignUp.tsx

```
// 회원 가입 폼 데이터 타입 정의
interface SignUpFormData {
  username: string;
  password: string;
  confirmPassword: string;
  email: string;
}

const SignUp = () => {
  const [formData, setFormData] = useState<SignUpFormData>({
    username: '',
    password: '',
    confirmPassword: '',
    email: '',
  });

  const navigate = useNavigate();

  // 폼 입력값 변경 핸들러
  const handleInputChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const { name, value } = e.target;
    setFormData({ ...formData, [name]: value });
  }
}
```



# 쇼핑몰 프로젝트

## ■ 회원 가입 구현 – users/SignUp.tsx

```
// 폼 제출 핸들러
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  // 입력값 추출
  const { username, password, confirmPassword, email } = formData;

  // 간단한 유효성 검사
  if (!username || !password || !confirmPassword || !email) {
    alert('모든 필드를 입력해주세요.');
```

```
    return;
  }

  if (password !== confirmPassword) {
    alert('비밀번호가 일치하지 않습니다.');
```

```
    return;
  }
  console.log('회원가입 시도:', formData);

  alert('회원가입이 완료되었습니다! 로그인 페이지로 이동합니다.');
```

```
  navigate('/signin'); // 회원가입 성공 후 로그인 페이지로 이동
}
```

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현 – users/SignUp.tsx

```
return (  
  <div className="signup">  
    <h2>회원 가입</h2>  
    <form onSubmit={handleSubmit}>  
      <div>  
        <label htmlFor="username">아이디 :</label>  
        <input  
          type="text"  
          id="username"  
          name="username"  
          value={formData.username}  
          onChange={handleInputChange}  
          placeholder='아이디를 입력하세요.'  
        />  
      </div>  
      <div>  
        <label htmlFor="email">이메일 :</label>  
        <input  
          type="email"  
          id="email"  
          name="email"  
          value={formData.email}  
          onChange={handleInputChange}  
          placeholder='이메일을 입력하세요.'  
        />  
      </div>  
    </form>  
  </div>  
)
```

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현 – users/SignUp.tsx

```
<div>
  <label htmlFor="password">비밀번호:</label>
  <input
    type="password"
    id="password"
    name="password"
    value={formData.password}
    onChange={handleInputChange}
    placeholder='비밀번호를 입력하세요.'
  />
</div>
<div>
  <label htmlFor="confirmPassword">비밀번호 확인:</label>
  <input
    type="password"
    id="confirmPassword"
    name="confirmPassword"
    value={formData.confirmPassword}
    onChange={handleInputChange}
    placeholder='비밀번호를 다시 입력하세요.'
  />
</div>
<button type="submit">회원가입</button>
</form>
</div>
```

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현 – App.css

```
/* 회원가입 스타일 */
.signup {
  width: 400px;
  margin: 20px auto;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.signup h2 {
  text-align: center;
  margin-bottom: 20px;
  color: #333;
}
.signup form {
  display: flex;
  flex-direction: column;
  gap: 15px;
}
.signup label {font-size: 16px; color: #555;}
```

# 쇼핑몰 프로젝트

## ■ 회원 가입 구현 – App.css

```
.signup input {  
  padding: 8px;  
  border: 1px solid #ccc;  
  border-radius: 4px;  
}  
.signup button {  
  padding: 10px;  
  background-color: #640aec;  
  color: white;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}  
.signup button:hover {background-color: #4b00b5;}
```

# 쇼핑몰 프로젝트

## ■ 관리자 대시보드

[Home](#) [상품 목록](#) [상품 등록](#) [admin님](#) [로그아웃](#)

### 관리자 DashBoard

#### 회원 현황

| ID | 아이디   | 역할    |
|----|-------|-------|
| 1  | admin | admin |
| 2  | user1 | user  |
| 3  | user2 | user  |

#### 상품 주문 현황

| 주문 ID | 상품명     | 주문자   | 상태   |
|-------|---------|-------|------|
| 1     | 모니터     | user1 | 배송중  |
| 2     | 무선마우스   | user2 | 배송완료 |
| 3     | USB 메모리 | user1 | 결제완료 |

# 쇼핑몰 프로젝트

## ■ 관리자 대시보드 – data/orders.json

```
[  
  {  
    "id": 1,  
    "userId": 2,  
    "product": "모니터",  
    "quantity": 1,  
    "totalPrice": 2000000,  
    "status": "배송중"  
  },  
  {  
    "id": 2,  
    "userId": 3,  
    "product": "무선마우스",  
    "quantity": 2,  
    "totalPrice": 64000,  
    "status": "배송완료"  
  },  
]
```

```
{  
  "id": 3,  
  "userId": 2,  
  "product": "USB 메모리",  
  "quantity": 3,  
  "totalPrice": 45000,  
  "status": "결제완료"  
}
```

# 쇼핑몰 프로젝트

## ■ 관리자 대시보드 – users/DashBoard.tsx

```
import users from '../data/users.json'
import orders from '../data/orders.json'

const DashBoard = () => {
  /* useLocation()은 현재 URL의 위치(location) 정보를 반환합니다.
   대시보드로 전달된 사용자 정보를 location.state에서 추출합니다.
   navigate('/dashboard', { state: { username: user.username, role: user.role } })
   role === 'admin'일 때만 관리자 테이블을 렌더링하는 구조입니다.
  */
  const location = useLocation()
  //{} - state가 null인 경우 안전 장치
  const { username, role } = location.state || {}

  console.log(username, role); // 전달된 사용자 정보 확인

  return (
    <div className="dashboard">
      <h2>관리자 DashBoard</h2>
      {role === 'admin' && (
        <>
          <div>
            <h3>회원 현황</h3>
```



# 쇼핑몰 프로젝트

## ■ 관리자 대시보드 – users/DashBoard.tsx

```
<table className="user-table">
  <thead>
    <tr>
      <th>ID</th>
      <th>아이디</th>
      <th>역할</th>
    </tr>
  </thead>
  <tbody>
    {users.map(user => (
      <tr key={user.id}>
        <td>{user.id}</td>
        <td>{user.username}</td>
        <td>{user.role}</td>
      </tr>
    ))}
  </tbody>
</table>
</div>

<div>
  <h3>상품 주문 현황</h3>
  <table className="order-table">
    <thead>
```

# 쇼핑몰 프로젝트

## ■ 관리자 대시보드 – users/DashBoard.tsx

```
<tr>
  <th>주문 ID</th>
  <th>상품명</th>
  <th>주문자</th>
  <th>상태</th>
</tr>
</thead>
<tbody>
  {orders.map(order => (
    <tr key={order.id}>
      <td>{order.id}</td>
      <td>{order.product}</td>
      {/* order.userId에 해당하는 유저가 존재하지 않을 수 있으므로
       optional chaining 연산자 사용, 런타임 에러 방지 */}
      <td>{users.find(user => user.id === order.userId)?.username}</td>
      <td>{order.status}</td>
    </tr>
  ))}
</tbody>
</table>
</div>
```

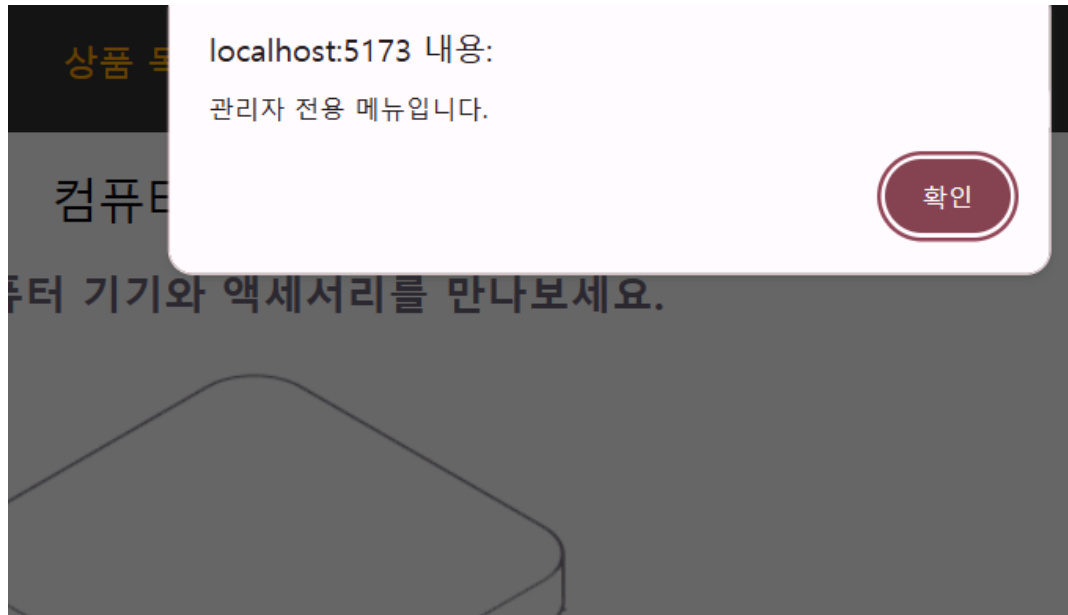
# 쇼핑몰 프로젝트

## ■ 관리자 대시보드 – App.css

```
/* Dashboard 스타일 */
.dashboard {
  width: 80%;
  margin: 20px auto;
  padding: 20px;
  border: 1px solid #ccc;
  border-radius: 5px;
}
.dashboard h2 {margin-bottom: 20px; color: #333;}
.dashboard table {width: 100%; border-collapse: collapse;}
.dashboard th, .dashboard td {
  border: 1px solid #ccc;
  padding: 8px;
  text-align: left;
}
.dashboard th {background-color: #f2f2f2; color: #333;}
```

# 쇼핑몰 프로젝트

- 상품 등록은 관리자 전용 메뉴



# 쇼핑몰 프로젝트

- 상품 등록은 관리자 전용 메뉴

[Home](#) [상품 목록](#) [상품 등록](#) admin님 [로그아웃](#)

### 상품 등록

상품명

상품명을 입력하세요.

가격

0

설명

설명을 입력하세요.

이미지

파일 선택

선택된 파일 없음

등록

# 쇼핑몰 프로젝트

## ■ 상품 등록은 관리자 전용 – App.tsx

```
function App() {  
  // 로그인 상태 관리  
  const [isLoggedIn, setIsLoggedIn] = useState(false);  
  
  // 로그인한 사용자 ID 관리 (예: username)  
  const [userId, setUserId] = useState<string | null>(null);  
  
  // 로그인한 사용자 권한 관리  
  const [userRole, setUserRole] = useState<string | null>(null);  
  
  // 로그인 상태 업데이트 함수  
  const handleLogin = (username: string, role: string) => {  
    setIsLoggedIn(true); // 로그인 상태를 true로 설정  
    setUserId(username); // 로그인한 사용자 ID 저장  
    setUserRole(role); // 로그인한 사용자 권한 저장  
  }  
  
  // 로그아웃 함수  
  const handleLogout = () => {  
    setIsLoggedIn(false);  
    setUserId(null);  
    setUserRole(null);  
  }  
}
```

# 쇼핑몰 프로젝트

## ■ 상품 등록은 관리자 전용 – App.tsx

```
return (  
  <>  
    <section className="app">  
      <BrowserRouter>  
        <Header  
          isLoggedIn={isLoggedIn}  
          userId={userId}  
          userRole={userRole}  
          onLogout={handleLogout}  
        />  
        <Routes>  
          <Route path="/" element={<Home />} />  
          <Route path="/products" element={<ProductList />} />  
          <Route path="/products/:id" element={<ProductInfo />} />  
          <Route path="/products/add" element={<AddProduct />} />  
          <Route path="/signin" element={<SignIn onLogin={handleLogin} />} />  
          <Route path="/signup" element={<SignUp />} />  
          <Route path="/dashboard" element={<DashBoard />} />  
        </Routes>  
      </BrowserRouter>  
    </section>  
  </>  
)
```

# 쇼핑몰 프로젝트

## ■ 상품 등록은 관리자 전용 – Header.tsx

```
// Header 컴포넌트 Props 타입 정의
interface HeaderProps {
  isLoggedIn: boolean;
  userId?: string | null;
  userRole?: string | null;
  onLogout: () => void;
}

const Header = ({ isLoggedIn, userId, userRole, onLogout }: HeaderProps) => {
  const navigate = useNavigate();

  return (
    <header className="header">
      <nav>
        <NavLink to="/">Home</NavLink>
        <NavLink to="/products">상품 목록</NavLink>
        <NavLink to="/products/add" onClick={(e) => {
          if (userRole !== 'admin') {
            e.preventDefault(); // 기본 링크 이동을 막고 경고 메시지를 표시
            alert('관리자 전용 메뉴입니다.');
```



# 쇼핑몰 프로젝트

## ■ 상품 등록은 관리자 전용 – SignIn.tsx

```
// 로그인 컴포넌트 Props 타입 정의
interface SignInProps {
  login: (username: string, role: string) => void;
}
```

```
console.log('로그인 시도:', formData)
login(user.username, user.role); // 로그인 성공 시 부모 컴포넌트에 알림

// 로그인 성공 시 권한에 따라 다른 페이지로 이동
if (user.role === 'admin') {
  setLoginResult('success') // 로그인 성공 상태 업데이트
  // 관리자 로그인 시 대시보드로 이동하면서 사용자 정보 전달
  navigate('/dashboard', { state: { username: user.username, role: user.role } })
} else {
  setLoginResult('success')
  navigate('/products')
}
```

# 리엑트 앱 배포

- Github Pages에 배포 전 사전 작업

1. shopping 앱을 미리 깃허브 저장소에 업로드 한다.
2. Settings > Pages 설정하기

1) **Settings** → **Pages**

2) **Build and deployment**:

- Source: **\*\*Deploy from a branch\*\***
- Branch: **\*\*main\*\***를 선택
- Folder: **\*\*/ (root)\*\*** 선택

3) **Save**

# 깃허브 Pages에 배포

- 배포시 경로 설정 – App.tsx

✓ `basename = "리포지터리 저장소명"`

```
<BrowserRouter basename='/shopping-test/'>
  <Header
    isLoggedIn={isLoggedIn}
    userId={userId}
    userRole={userRole}
    onLogout={handleLogout}
  />
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/products" element={<ProductList />} />
    <Route path="/products/:id" element={<ProductInfo />} />
    <Route path="/products/add" element={<AddProduct />} />
    <Route path="/signin" element={<SignIn onLogin={handleLogin} />} />
    <Route path="/signup" element={<SignUp />} />
    <Route path="/dashboard" element={<DashBoard />} />
  </Routes>
</BrowserRouter>
```

# 깃허브 Pages에 배포

- package.json

✓ Homepage, predeploy, deploy 속성 추가

```
{
  "name": "component",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "homepage": "https://kiyongee2.github.io/shopping-test/",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "predeploy": "npm run build",
    "deploy": "gh-pages -d dist"
  },
  "dependencies": {
    "react": "^19.2.4",
```

# 깃허브 Pages에 배포

- vite.config.js

- ✓ base: '/리포지터리 저장소 이름/'

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  base: 'https://kiyongee2.github.io/shopping-test/',
})
```

# 깃허브 Pages에 배포

- index.html

✓ 상대 경로로 수정

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>component</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="./src/main.tsx"></script>
  </body>
</html>
```

# 깃허브 Pages에 배포

- 필수 라이브러리 설치 및 빌드

1. `npm install`
2. `npm install gh-pages --save-dev`
3. `npm run build`

# 깃허브 Pages에 배포

- 수정된 파일 업로드 후 deploy(배포)

1. `git add .`
2. `git commit -m "deploy"`
3. `git push origin main`
4. `npm run deploy`



# 깃허브 Pages에 배포

- 수정된 파일 업로드 후 deploy(배포)

## 1) Build and deployment:

- Source: **\*\*Deploy from a branch\*\***
- Branch: **\*\*gh-pages\*\***를 선택
- Folder: **\*\*/ (root)\*\*** 선택

## 2) Save

Your site is live at <https://kiyongee2.github.io/shopping-test/>

Last [deployed](#) by  [kiyongee2](#) 25 minutes ago

 Visit site

Unpublish site

# 404 오류 문제

- 404 오류 – File Not Found

**문제:** 새로고침 시 /shopping-test/products 같은 경로로 직접 접근하면 GitHub Pages가 해당 파일을 찾으려고 하는데, 실제 파일이 없어서 404가 발생합니다.

**해결방법:** public/404.html을 생성하여 index.html과 동일한 내용으로 설정하면, GitHub Pages가 모든 존재하지 않는 경로를 자동으로 index.html로 라우팅합니다.

# 404 오류 문제

## ▪ 404.html – 아직 오류 미해결

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>router</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="./src/main.jsx"></script>
  </body>
</html>
```

문제는 [404.html](#)의 스크립트 경로입니다. Vite 빌드 후 [404.html](#)은 단순 복사되므로, [main.jsx](#) 경로가 무효입니다.

가장 간단한 해결책은 비트 빌드 후 자동으로 [index.html](#)을 [404.html](#)로 복사하도록 설정하는 것입니다:

# 404 오류 문제

## ▪ vite.config.js 설정 변경

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import fs from 'fs'
import path from 'path'

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    react(),
    {
      name: 'copy-404',
      writeBundle() {
        // 빌드 후 dist/index.html을 dist/404.html로 복사
        const distPath = path.resolve(__dirname, 'dist')
        const indexPath = path.join(distPath, 'index.html')
        const notFoundPath = path.join(distPath, '404.html')
        if (fs.existsSync(indexPath)) {
          fs.copyFileSync(indexPath, notFoundPath)
        }
      }
    }
  ],
  base: '/shopping-test/'
})
```

추가